Choosing Agentic Architectures

Mode 2 Phase 3 (Scale the Workforce), Chapter 8/9 — Revision & Viva-Prep Cheatsheet (English + Roman Urdu)

Source: agentfactory.panaversity.org/docs/choosing-agentic-architectures-crash-course

EN: Pattern selection is architectural FIT, not capability matching -- choose the simplest pattern the task demands, add complexity only on real demand. 5 questions deterministically pick a starting pattern.
Roman Urdu: Pattern selection architectural FIT hai, capability matching nahi -- simplest pattern chuno jo task maangta, complexity sirf real demand par add karo. 5 sawal deterministically starting pattern decide karte.

1. The Five-Question Decision Tree

- Q1: Is the path definable (could you write a plain Python function without an LLM)? Q2: Is it fixed/stable? -> Sequential Workflow. Q3: Is the structure articulable (stages known, content not)? -> Planning+ReAct, else Single Agent+ReAct.
- Q4: Quality>speed AND checkable? -> add a Reflection layer (critic must differ from generator). Q5: Genuine specialization/context/scale bottleneck? -> add Multi-Agent. (Q1-Q3 pick the core pattern; Q4-Q5 are additive layers.)

2. Two Failure Modes

- **Overshooting:** an elaborate pattern (usually multi-agent) nobody needed -- famous and visible, but hard to undo (a rewrite, not a refactor).
- **Undershooting:** simpler than needed -- less discussed, more dangerous, silently brittle. Both are equally important failures.

3. Five Patterns In Depth

- Sequential Workflow (narrow typed Agents, smallest cloud stack). Single Agent+ReAct+Tools (full cloud stack, reasoning-trace evals). Planning+ReAct (plan persisted, biggest crash-savings).
- Reflection (additive; rubber-stamping is the hardest failure to detect -- measure net-positive, not 'does it run'). Multi-Agent (3 scoreboards: specialist quality x routing x integration compound multiplicatively).

4. Failure Signals + Cost

- 5 revision signals: loops/revisits, plan-execution divergence, reflection not improving, routing failures, complex-but-not-better. Try the cheapest fix (prompt/contract) BEFORE an architectural change.
- Cost multiplier table: Sequential ~1x -> Multi-agent+Reflection 30-60x. Cost/latency are architectural constraints, not afterthoughts.

Key Terms Cheatsheet

One line each — English + Roman Urdu.

Term	English	Roman Urdu
Fit-matching, not capability-matching	Choose the simplest pattern the task demands, not the most impressive one.	Simplest pattern chuno jo task maange, sabse impressive nahi.
Overshooting vs Undershooting	Elaborate-but-unneeded (visible, hard to undo) vs too-simple (quiet, brittle).	Elaborate-but-unneeded (visible, undo mushkil) vs too-simple (quiet, brittle).
Reflection layer	An additive critic-refiner loop -- rubber-stamping is its hardest failure to catch.	Additive critic-refiner loop -- rubber-stamping iska sabse mushkil failure pakarna.
3 compounding scoreboards (Multi-Agent)	Specialist quality x routing x integration -- multiply, don't just average.	Specialist quality x routing x integration -- multiply karte, average nahi.
Cost multiplier	Sequential ~1x vs Multi-agent+Reflection 30-60x the cost for the same task.	Sequential ~1x vs Multi-agent+Reflection 30-60x cost, same task ke liye.

Quick Self-Check

Question	Answer
What's the most common architectural mistake this course names?	Overshooting -- defaulting to an elaborate, impressive-looking pattern (often multi-agent) when a simpler one would do.
What are the 2 conditions required before adding a Reflection layer (Q4)?	Quality matters more than speed, AND there's a checkable criterion -- plus the critic must genuinely differ from the generator.
Why is rubber-stamping the hardest Reflection failure to detect?	The loop still 'runs' successfully -- you must measure whether it's actually net-positive, not just whether it executes.
Why do Multi-Agent systems need 3 separate scoreboards instead of one?	Specialist quality, routing accuracy, and integration quality compound multiplicatively -- a single aggregate score hides which layer is actually failing.

Where This Leads / Sources

Leads to: Payment-Enabled Agents (final Mode 2 chapter). Full Decision Lab (5 worked examples) + Design-Review Template in Part 5.

Source: docs/choosing-agentic-architectures-crash-course/ (this repo), content originally via Zia Tutor AI connector.